

part C - [derive attention gradients by hand]

setup: • $S = \frac{QK^T}{\sqrt{d_k}}$ (unscaled then scaled scores, shape $T \times T$)

- $P = \text{softmax}(S)$ (row-wise softmax, shape $T \times T$)
- $A = P \mathbf{V}$ (attention output, shape $T \times d_v$)

Let L be scalar loss. In standard backpropagation when we write $\frac{\partial A}{\partial v}$, we are typically solving for

how the upstream gradient $\frac{\partial L}{\partial A}$ flows into v ,

which we denote as $dV = \frac{\partial L}{\partial v}$.

① derive $\frac{\partial A}{\partial v}$.

$$A_{ij} = \sum_k P_{ik} v_{kj} \quad \text{--- ①}$$

taking partial derivative of A_{ij} wrt specific element v_{kj} :

$$\frac{\partial A_{ij}}{\partial v_{kj}} =$$

$$\frac{\partial L}{\partial V_{kj}} = P_{ik} \quad \text{--- (2)}$$

applying chain rule using the upstream gradient dA (where $dA_{ij} = \frac{\partial L}{\partial A_{ij}}$)

$$dW_{kj} = \frac{\partial L}{\partial V_{kj}} = \sum_i \frac{\partial L}{\partial A_{ij}} \cdot \frac{\partial A_{ij}}{\partial V_{kj}} = \sum_i dA_{ij} P_{ik}$$

$$dW = P^T \cdot dA$$

$$\frac{\partial A}{\partial V} = \nabla V = P^T (\nabla_A)$$

② softmax Jacobian

We want to find $\frac{\partial p_i}{\partial s_j}$ for the softmax fn

$$p_i = \frac{e^{s_i}}{\sum_k e^{s_k}}$$

$$\sum_k e^{\eta_k}$$

Case 1 $i=j$ - (derivative wrt its own logit)
using the quotient rule $\rightarrow$

$$\frac{\partial p_i}{\partial s_i} = \frac{\frac{\partial}{\partial s_i} (e^{\eta_i}) \cdot \sum - e^{s_i} \frac{\partial}{\partial s_i} \sum}{\sum^2}$$

$$= \left(\frac{e^{s_i}}{\sum} \right) - \left(\frac{e^{s_i}}{\sum} \right)^2$$

$$\boxed{\frac{\partial p_i}{\partial s_i} = p_i - p_i^2 = p_i(1-p_i)}$$

Case 2 $i \neq j$ $\frac{\partial}{\partial s_j} (e^{s_i}) = 0$

$$\frac{\partial p_i}{\partial s_j} = \frac{0 - e^{s_i} e^{s_j}}{\sum^2} = \left(-\frac{e^{s_i}}{\sum} \right) \left(\frac{e^{s_j}}{\sum} \right)$$

$$= \underline{\underline{-p_i p_j}}$$

combining these,

$$\frac{\partial p_i}{\partial s_j} = p_i (s_{ij} - p_j)$$

③ Derive $\frac{\partial A}{\partial Q}$.

We use the chain rule to flow the gradient dA backward through P , then S , to Q .

gradient through P from $A = PV$, the gradient dP (shape $T \times T$) is:

$$(2) \quad dP = dA \cdot V^T$$

gradient through P

(b) gradient through S . Using the softmax Jacobian derived in step 2, we calculate dS_{ij} . The softmax is applied rowwise, so the gradient for a single element

S_{ij} depends on the entire i -th row of P .

$$dS_{ij} = \sum_k dP_{ik} \frac{\partial P_{ik}}{\partial S_{ij}}$$

substitute the Jacobian;

$$dS_{ij} = \sum_k dP_{ik} P_{ik} (\delta_{kj} - P_{ij})$$

distribute:

$$dS_{ii} = dP_{ii} P_{ii} (1 - P_{ii}) + \sum_{j \neq i} dP_{ij} P_{ij} (0 - P_{ii})$$

$$-s_{ij} = -\sum_{k \neq j} \frac{p_{ik} p_{kj}}{p_{ij}}$$

$$dS_{ij} = dP_{ij} P_{ij} - \sum_k dP_{ik} P_{ik} P_{ij}$$

factor out P_{ij} :

$$dS_{ij} = P_{ij} \left(dP_{ij} - \sum_k dP_{ik} P_{ik} \right)$$

(c) Gradient through Q

We know $S = \frac{QK^T}{\sqrt{d_k}}$. In Index notation,

$$S_{ij} = \frac{1}{\sqrt{d_k}} \sum_m Q_{im} K_{jm}$$

$$dQ_{im} = \sum_j dS_{ij} \frac{K_{jm}}{\sqrt{d_k}} \quad ; \text{ which in matrix form is}$$

$$dQ = dS \cdot \frac{K}{\sqrt{d_k}}$$

final combined answer -

$$\nabla Q = ds \cdot \frac{k}{\sqrt{d_k}}$$

4. Interpretation

Why does the gradient through softmax become very small when logits have large magnitudes?

When the input logits (S) have very large magnitudes, the exponentiation in the softmax function causes the largest logit to completely dominate the sum. As a result, the output probability distribution P becomes highly peaked: the largest value approaches 1.0, and all other values approach 0.0.

Looking at the Softmax Jacobian we derived: $\frac{\partial p_i}{\partial s_j} = p_i(\delta_{ij} - p_j)$.

- If $p_i \approx 1$, then $(1 - p_i) \approx 0$, so the diagonal gradient $p_i(1 - p_i) \rightarrow 0$.
- If $p_i \approx 0$, then p_i multiplied by anything is 0, so both the diagonal and off-diagonal gradients $(-p_i p_j) \rightarrow 0$.

Therefore, when logits are large, every entry in the Jacobian matrix approaches zero. The gradient vanishes, and the network stops learning.

Connection to $\sqrt{d_k}$: The pre-softmax logits are calculated via a dot product QK^T . If Q and K vectors have a variance of 1, their dot product across d_k dimensions will have a variance of d_k . For large embedding dimensions, this causes naturally massive logits. Dividing by $\sqrt{d_k}$ standardizes the variance back to 1, keeping the logits small and ensuring the softmax operates in its "soft," differentiable region where gradients can flow.